lab 1 RISC-V Matrix-Matrix Multiplication Lab

Start Assignment

- Due Thursday by 5pm
- Points 100
- Submitting a file upload

Objective:

This lab aims to implement a matrix-matrix multiplication algorithm in assembly language for the RISC-V architecture. This exercise will help you understand the concepts of matrix multiplication and assembly language programming for RISC-V.

Requirements:

- Basic understanding of RISC-V assembly language.
- Familiarity with matrix multiplication algorithms.
- RISC-V assembly development environment

Resources:

- **[/courses/186931/files/21847955?wrap=1 (https://canvas.calpoly.edu/courses/186931/files/21847955?wrap=1)](https://canvas.calpoly.edu/courses/186931/files/21847955?wrap=1)**
  - Starting at page 326 goes over how to store variables on the stack (leaf vs non-leaf functions/subroutines)

Background:

Matrix multiplication is a fundamental operation in linear algebra and finds applications in various fields, such as computer graphics, scientific computing, and machine learning. The algorithm for matrix-matrix multiplication involves multiplying each element of a row from the first matrix with each element of a column from the second matrix and summing up the results to obtain the corresponding component of the resulting matrix.

Consider two matrices A and B:

A = | a11 a12 a13 | B = | b11 b12 b13 |
   | a21 a22 a23 | | b21 b22 b23 |
   | a31 a32 a33 | | b31 b32 b33 |

The resulting matrix C, obtained by multiplying A and B, will be:

C = | (a11*b11 + a12*b21 + a13*b31) (a11*b12 + a12*b22 + a13*b32) (a11*b13 + a12*b23 + a13*b33) |
   | (a21*b11 + a22*b21 + a23*b31) (a21*b12 + a22*b22 + a23*b32) (a21*b13 + a22*b23 + a23*b33) |
   | (a31*b11 + a32*b21 + a33*b31) (a31*b12 + a32*b22 + a33*b32) (a31*b13 + a32*b23 + a33*b33) |

This code in C maybe useful to understand the implementation:

```c
for (int i = 0; i < R1; i++) {
    for (int j = 0; j < C2; j++) {
        result[i][j] = 0;

        for (int k = 0; k < R2; k++) {
            result[i][j] += m1[i][k] * m2[k][j];
        }

        printf("%d\t", result[i][j]);
    }

    printf("\n");
}
```

Procedure:

1. **Initialize Matrices:** Define A, B, and C with appropriate dimensions and initial values.
2. Create a **FUNCTION** to perform **Matrix-Matrix Multiplication** Algorithm using the stack correctly and the RISC-V

calling convention as explained in class. NOTE: OTTER doesn't have a mult instruction so create a function to do this with add and a for loop. Assume the values in the matrices will all be positive (including 0).

3. **Call the function** you just created with the initialized Matrices; you can find initialization for 3x3,...,50x50 arrays in datasets.zip (https://canvas.calpoly.edu/courses/186931/files/21847919?wrap=1) (https://canvas.calpoly.edu/courses/186931/files/21847919/download?download_frd=1)
4. **Check** that the implementation is correct by comparing your results with the values generated by a matrix multiplication calculator like https://matrix.reshish.com/multiplication.php (https://matrix.reshish.com/multiplication.php)

UPLOAD

1. Report: pdf with names of 2-4 project members and describe each person's contribution and number of hours spend on the project by each member.

2. Source code as a zip file. You can copy paste your code on the pdf document also but upload a zip file with source *.asm so I can test your implementation

3. Show that your implementation works by comparing the results you are producing vs the ones produced on step 4 above and copy paste a screen capture of the correct (equal) results.